What's Actually In the Models You Pull? A Registry-Wide Audit of Behavioral Supply-Chain Integrity in the Consumer GGUF Ecosystem

Anonymous Author(s)
Anonymous Institution

Abstract

A GGUF file downloaded from Hugging Face carries more than quantized weights: it embeds a Jinja2 chat template, a default system prompt, and sampling parameters that determine how a prompt actually reaches the model. None of this metadata is verified against the model developer's canonical release, by the registry, by the loader, or in practice by the user, and a May 2026 incident in which a broken template shipped registry-wide with early Gemma-4 conversions shows the consequences are not hypothetical. This paper presents the first registry-scale measurement of how often that metadata diverges from what a model's developer actually shipped, and whether the divergence changes model behavior. Auditing 143 Hugging Face repositories across five anchor model families, with canonical ground truth independently verified for three, we find 34.5% of canonical-verified repositories diverge from their developer's template. A second, independent corpus built against ModelScope for the same three verified anchors confirms the phenomenon replicates outside Hugging Face's specific ecosystem (60.4% pooled divergence, a different but consistently substantial magnitude). Divergence is not concentrated in the unpopular long tail, as commonly assumed; popularity is not a reliable signal in either direction, and for one anchor (Mistral) the popular repositories are the more divergent group, a pattern traced to ecosystem-wide propagation lag rather than publisher negligence. Behavioral testing across all five anchors confirms one statistically significant safety-relevant effect (+35.7 percentage points), one real but underpowered effect, two ceiling effects bounded by the model's own baseline safety training, and one anomaly resolved analytically as behaviorally inert. We conclude with practitioner guidance for registries and loaders on verifying template fidelity before serving a model.

1 Introduction

Measure a model's safety behavior and you're implicitly assuming the weights in front of you are the weights the developer shipped, wrapped in the template the developer intended. Leave the developer's own repository and that assumption stops holding. A GGUF file downloaded from Hugging Face, the dominant format for running open-weight models locally via llama.cpp, LM Studio, and dozens of other loaders, carries more than quantized weights: a Jinja2 chat template, a default system prompt, sampling parameters. Nobody checks any of it against the developer's canonical release. Not the registry, not the loader, not, in practice, the user. Whoever quantized and republished the model set it, and it is the layer that actually decides what the model does when a prompt reaches it.

In May 2026 this stopped being a hypothetical concern. A broken chat template shipped registry-wide with early Gemma-4 GGUF conversions. The template failed to render correctly across a swath of community repositories before being patched, and it affected users regardless of which specific quantizer's repository they happened to pull from. This was not one publisher's isolated mistake. It was a structural property of an ecosystem in which a template can propagate error at registry scale before anyone notices, because nothing routinely checks it against the developer's own release. A separate, earlier line of user reports on Hugging Face's forums describes Ollama serving a simplified chat template that diverges from the canonical version for certain models. That report is worth citing as evidence that template-fidelity failures are a real, recurring pattern in this space, even though Ollama's own templating format (Go's `text/template`) is not the artifact this paper measures; Section 2 explains why. Together, these two incidents motivate the question this paper answers with data rather than anecdote: how often, and by how much, does the metadata layer of a locally-deployed model diverge from what its developer actually shipped, and when it does, does that divergence change what the model will do?

This paper's independent variable is the packaging intermediary, not the model developer. Holding a single model developer's release fixed, it asks whether the community quantization ecosystem, the publishers, registries, and loaders sitting between what the developer trained and what the user

runs, faithfully reproduces that developer's own chat template or silently diverges from it. This is a deliberately narrow question. It makes no claim about whether any model developer's safety training is adequate, well-designed, or comparable to another developer's; it asks only whether what a user actually receives matches what that developer released.

Contributions. This paper makes four. First, a static-audit method that extracts and diffs embedded GGUF chat-template metadata at registry scale, validated against ground-truth canonical templates rather than assumed majority consensus. Second, the first prevalence measurement of publisher-introduced template divergence in the consumer GGUF ecosystem, stratified by model family and by repository popularity. Third, a behavioral-confirmation methodology connecting flagged static anomalies to measured safety-relevant consequences rather than reporting structural difference alone, tested across all five anchor model families in the corpus. Fourth, practitioner guidance for registries and loaders on what should be verified before a GGUF is served to a user.

Scope. This paper measures publisher-introduced divergence in locally-deployed model metadata. It does not measure or claim anything about a model developer's safety training itself; whether one developer's safety training is more robust than another's is a distinct research question this paper does not address. It is not a demonstration of a new attack primitive: chat-template prompt injection and GGUF-based server-side template injection are already documented in the literature (Section 3). It is also not a code-execution prevalence study, a related but distinct vulnerability class already measured elsewhere. What is missing from the literature, and what this paper supplies, is a measurement of how often the metadata layer itself silently diverges from what the developer shipped, and whether that divergence is behaviorally consequential.

2 Threat Model and Background

What a chat template is, and why it matters. A GGUF file bundles quantized model weights together with metadata the inference runtime needs to actually talk to the model correctly: a Jinja2 template describing how to format a conversation, including where or whether a system-level instruction is inserted, a default system prompt, and default sampling parameters. Two GGUF files can contain byte-identical weights and still behave differently, because the template controls whether and how safety-relevant instructions reach the model at inference time. This metadata sits inside a binary header most end-user tooling never surfaces. A user selecting a model in llama.cpp or LM Studio sees a file size and a quantization level, not a diff against the developer's own template.

The registry blind spot. Hugging Face's own model API (`expand[]=gguf`) reports metadata for a single file per repository, typically the first or most prominent quantization level, even when a repository hosts many GGUF variants with potentially different embedded templates. A registry-level scan that trusts this API inherits a documented "first file clean, subsequent files unaudited" gap. This paper's static-audit tooling (Section 4) routes around that limitation via direct HTTP Range requests against each file's own binary header, rather than trusting the registry's own summary.

Four categories of divergence, not three. An earlier draft of this threat model separated divergence into three causes: honest publisher error, a registry or tooling structural blind spot, and adversarial intent. A fourth category is necessary and turns out to matter empirically: upstream checkpoint or template drift, in which the model developer itself revises its own canonical chat template over time, and downstream quantizations made before that revision then appear divergent for reasons having nothing to do with the publisher who made them. The May 2026 Gemma-4 incident is an instance of the second category, a registry or tooling blind spot, not the first, third, or fourth. But as Section 5 shows, the fourth category is not merely a theoretical addition to this taxonomy; it explains the majority of one anchor model's measured divergence. This paper's static-audit method can detect the first three categories directly, by comparing a repository's embedded template against the ground-truth canonical version and, where feasible, against upload timestamps that bear on whether a divergent template predates or postdates a known revision. It can only flag candidates for the fourth category, adversarial intent. It cannot establish intent from metadata alone and does not claim to.

Why Ollama sits outside this paper's primary measurement. Ollama, one of the most widely used tools for running local LLMs, does not consume the Jinja2 `chat_template` this paper's method extracts. It uses its own Go `text/template`-based Modelfile format, a structurally different templating language that this paper's tooling cannot directly hash-compare against the Jinja2 population without building a separate semantic-equivalence layer, which this project did not undertake. A user who runs `ollama pull` never encounters the specific divergence this paper measures. A second, distinct point is worth being precise about too: Ollama's Modelfile format can carry its own independent system-prompt and parameter overrides, layered on top of whatever the underlying GGUF's own embedded metadata specifies. That is a second divergence surface, Modelfile versus GGUF, that this paper does not audit at all, separate from the template-language difference already noted. The two surfaces are not fully independent: when a user runs a GGUF directly through Ollama without authoring a custom Modelfile, Ollama selects a Go-template match automatically based

on the GGUF's own embedded `tokenizer.chat_template` metadata, rather than either ignoring that metadata entirely or executing it directly [4]. This means the specific Jinja2 template a publisher embedded, the exact artifact this paper measures, can influence which Go template Ollama's own matching process selects for a bare GGUF import, even though Ollama never executes the Jinja2 template itself. This is a real, if partial, coupling between the two divergence surfaces, not evaluated further here. Ollama's five official registry entries were fetched as part of this project's corpus and are reported as a secondary reference point (Section 4.3), not pooled into the primary Jinja2-ecosystem results.

3 Related Work

Three distinct lines of prior work bear on this paper, and none of them occupies the specific intersection this paper measures: registry scale, GGUF-specific chat-template metadata, a point-in-time snapshot, and a behavioral rather than purely structural claim of consequence.

The attack and proof-of-concept line. Two independent efforts establish that the chat-template layer is exploitable, not one. Fogel et al. [2] demonstrate inference-time backdoors delivered via hidden instructions in chat templates, tested across eighteen models spanning seven families and four inference engines, specifically targeting GGUF-format models and their embedded Jinja2 templates on Hugging Face. Under a trigger condition, factual accuracy in their study drops from roughly 90% to 15% on average, and attacker-controlled URLs are emitted at success rates above 90%. This is a single study appearing in two places (an arXiv preprint and a company writeup on the same authors' own blog); citing both as if they were independent corroboration would overstate the evidence, so this paper treats them as one source. Separately, JFrog's GGUF-SSTI disclosure [5] documents CVE-2024-34359 ("Llama Drama"): unsandboxed Jinja2 rendering of a GGUF's embedded template enabling remote code execution in vulnerable versions of `llama-cpp-python`. That is a code-execution bug, not a behavioral-manipulation attack, and a different vulnerability class from Fogel et al.'s [2] work, even though both operate on the same underlying artifact: GGUF's `chat_template` field is the shared substrate for both concerns, conceptually distinct but not disjoint in the artifact they act on. Both efforts are hand-crafted, small-N demonstrations. Neither is a registry-scale measurement of how often the underlying condition actually occurs across the corpus of models people download. This paper does not introduce a new attack; it measures the prevalence of the underlying condition these two lines of work show can be weaponized.

The prevalence-measurement-of-a-different-vulnerability-class line. Zhao et al. [7], "Models Are Codes," conduct a registry-scale audit of malicious code poisoning across Hugging Face-hosted model formats. Its vulnerability class, arbitrary code execution via unsafe model-format deserialization, is orthogonal to the one measured here: a GGUF file's chat-template metadata is inert text interpreted by a template engine, not executable code loaded by a deserializer. A registry could patch every deserialization vulnerability Zhao et al. [7] describe and a downloaded model could still silently diverge from its developer's intended safety configuration; the two risks are additive, not substitutable. This paper borrows the registry-scale genre and applies it to a different, behavior-oriented axis.

The adjacent, orthogonal-axis line. Fan et al. [1], "The Moving Target," conduct a longitudinal audit of trustworthiness drift across twelve checkpoints of open-source chat models, tracking how a single developer-released model's properties change across its own official version history over time. This paper's baseline is fixed instead: given one developer's release at one point in time, how much does the community-republished ecosystem around it diverge from that single canonical release, measured once rather than tracked over successive versions? A registry could exhibit zero longitudinal drift in Fan et al.'s [1] sense while still showing substantial cross-registry divergence in this paper's sense. The two axes are complementary, not overlapping.

No published work sits at the intersection these three lines leave open: registry scale, GGUF-specific chat-template metadata, a point-in-time snapshot, and behavioral confirmation of consequence rather than structural difference alone. That intersection is this paper's contribution.

4 Method

4.1 Corpus construction

The corpus targets five anchor model families, Mistral-7B-Instruct-v0.3, Qwen2.5-7B-Instruct, Phi-3-mini-4k-instruct, Llama-3.1-8B-Instruct, and gemma-2-9b-it, chosen to span multiple developers, architectures, and release timelines. For each anchor, community GGUF quantizations were identified via Hugging Face's model-search API and filtered in two required stages: first by declared `base_model` metadata matching the anchor, then by a fine-tune-keyword exclusion list (`dpo`, `wpo`, `simpo`, `cpo`, `ipo`, `rrhf`, `rlaif`, `grpo`, and related terms) to remove preference-tuned derivatives.

Both stages turned out to be necessary. `base_model` metadata alone is insufficient: it records "built from X," not "identical to X," and a first pass of the corpus included DPO- and WPO-tuned derivatives that correctly declared their base model while being materially different fine-tunes. Even after adding the keyword filter, a second contamination case surfaced later: two SimPO-tuned repositories for gemma-2-9b-it slipped past the initial keyword list because `simpo` had not

Table 1: Corpus size for prevalence-estimate precision (95% CI).

Margin	n at $p = 0.229$	n at $p = 0.5$
±10pp	68	96
±7pp	138	196
±5pp	271	384

yet been added to it. Both entries were manually struck once found, and all downstream numbers reflect the corrected set. This filter is a heuristic requiring ongoing maintenance, not a one-time validated gate.

The final corpus contains 143 Hugging Face repositories across the five anchors, stratified by download count into a high-popularity bucket and a long-tail bucket, plus 5 official Ollama registry entries tracked separately (Section 4.3). Popularity stratification used each repository's Hugging Face download count as the ranking signal within its own anchor family: for each anchor, the top 15 genuine re-quantizations by download count were assigned to the high-popularity bucket, and the bottom 20 with nonzero downloads to the long-tail bucket, rather than one fixed global download threshold.

The corpus target was set via a standard single-proportion sample-size calculation, using this project's own pilot-stage observed prevalence (8/35 repositories, 22.9%). Table 1 shows the derivation across three candidate margins, alongside the conservative $p = 0.5$ estimate.

The ±7pp margin was chosen as the practical target, giving N≈150–200. The final corpus landed at N=143, a slight undershoot; the achieved margin (Wilson 95% CI: [25.1, 40.2] around 32.2%) is approximately ±7.5pp, close enough to the design target that the shortfall does not materially change what the paper can claim.

4.2 Static audit

Hugging Face's `expand[]=gguf` API reports metadata for one file per repository, inadequate for a corpus where a single repository may host several quantization levels with independently embedded templates. This paper's static-audit tooling instead parses each GGUF file's binary header directly via HTTP Range requests, without downloading the full weight file. The parser's output was validated by confirming it reproduces Hugging Face's own reported hash exactly on files where the platform's API does report metadata.

Two data-quality failures surfaced during the audit and are reported here rather than silently corrected. First, an initial full-corpus run silently failed on 101 of 150 attempted fetches due to Hugging Face API rate-limiting; the resulting topline of 43.2% divergence, computed from only 44 successfully fetched repositories, was wrong. This was caught by checking fetch-completeness before reporting any number, fixed with retry-with-backoff logic, and re-run with zero failures. Second, the SimPO-contamination case above was caught after the static audit had already run once, requiring recomputation of all downstream numbers.

Table 2: ModelScope corpus exclusion breakdown.

Exclusion reason	Count
Model-family variant mismatch	103
Not GGUF library	9
No chat_template in response	3
Fine-tune keyword match	1
Excluded / Included	**116 / 48**

4.3 Corpus scope statement, and a second registry for external validity

The primary corpus and every prevalence and behavioral result reported in Section 5 is drawn from Hugging Face-hosted repositories using the Jinja2 `chat_template` format. The 5 official Ollama registry entries are tracked as a separate, secondary reference point and never pooled into the primary analysis, because Ollama's Go `text/template` format is not hash-comparable against the Jinja2 population without a semantic diffing layer this project did not build.

A single-registry corpus carries an external-validity risk: Hugging Face's search API decides which repositories are discoverable at all, and a finding built entirely on what that one platform surfaces could be an artifact of that platform's community. A second, independent corpus was built against ModelScope (modelscope.cn, Alibaba's model hub) for the three anchors with verified canonical ground truth. ModelScope's search API exposes the embedded `chat_template` string directly. This endpoint is not part of ModelScope's officially documented public API surface; it was identified by inspecting the network requests the platform's own web frontend makes, and unlike Hugging Face's documented API, it could change or be withdrawn without notice.

It does not expose a reliable declared base-model field, so filtering relies on a strict name-prefix match plus an explicit exclusion list for same-name-prefix model-family variants, in addition to the fine-tune-keyword exclusion. Table 2 reports the exclusion breakdown: of 164 raw search hits across the three anchors, 116 were excluded (103 model-family variant mismatches, 9 missing the `gguf` library tag, 3 with no chat template exposed, 1 fine-tune-keyword match), leaving 48 included repositories.

4.4 Canonical-versus-modal divergence

An early version of this pipeline computed divergence from the modal (most common) template among a given anchor's quantizers, assuming the majority template is correct. That

Table 3: Behavioral-confirmation power calculation.

Effect	h	n (80%)	n (90%)
19pp	0.386	53	70
15pp	0.303	86	115
10pp	0.201	195	261
5pp	0.100	784	1,050

assumption is not safe: if the popular template is itself broken, a modal-based method inverts the paper's central causal claim. This gap was closed by fetching each anchor's true canonical `chat_template` directly from the developer's own repository. Three of five anchors (Mistral, Qwen2.5, Phi-3) are ungated and were fetched directly; the remaining two (Llama-3.1, gemma-2-9b-it) sit behind Hugging Face's gated-access mechanism, which this project did not bypass. No substitute proxy was used for these two; only modal-disagreement prevalence is reported for them, labeled explicitly as not canonical-verified.

For all three anchors where ground truth is available, the modal template turned out to be the canonical one, with no sign inversion.

4.5 Behavioral confirmation

For flagged anomalies, behavioral confirmation uses a HarmBench-derived prompt set [6] and a soft-refusal-aware REFUSE/COMPLY classifier built for this study, applied only to flagged anomalies, bounding compute regardless of corpus size N. "Behaviorally consequential" is defined operationally: an effect is treated as statistically confirmed only if Wilson 95% confidence intervals between compared conditions do not overlap, a conservative bar set before any pilot was run.

Any flagged anomaly receives a 2×2-style decomposition, isolating a content-delivery effect (does the safety-relevant instruction reach the model, holding template shape constant) from a template-shape effect (does structural verbosity alone shift behavior, holding delivered content constant). This decomposition, not a single before/after comparison, is what resolved the canonical-versus-modal risk: checking each pilot's tested arms directly against the verified canonical hash confirmed, for both anchors with a real effect, that the system-delivered arm exactly matches the canonical template and the no-system arm is the genuine divergence.

Sample sizes follow a pre-registered power calculation (Cohen's h). Table 3 reports the per-condition sample size needed at 80%/90% power for four target effect sizes. The initial n=21 budget for every anchor was set before any pilot data existed; the subsequent n=56 scale-up for Mistral and Phi-3 was an interim-analysis-informed increase using each anchor's own n=21 effect size as the planning input, a legitimate but distinct kind of pre-registration named as such rather than conflated with the first stage.

4.6 Instrument-validity check

Automated REFUSE/COMPLY classifiers of this kind are known in the safety-evaluation literature to under-detect soft or indirect refusals. Because this paper's Stage-2 pilots depend on this classifier's output for every reported effect size, a hand-validation pass was run: 45 of 252 total pilot generations were sampled across the four behaviorally-tested anchors (Mistral, Phi-3, gemma, Llama-3.1); 25 from Mistral, the anchor with the largest measured effect. Qwen2.5's divergence was resolved analytically (Section 4.8) with no pilot run, so there were no generations to sample from it. The overall error rate on the 44 non-ambiguous items checked was 25.0%, and every error was a false negative; zero false positives were found. Every reported refusal rate in this paper's Stage-2 conditions should therefore be read as a lower bound.

4.7 Responsible disclosure

Any concrete, individually attributable safety-relevant anomaly discovered during this audit would be reported to the specific publisher or registry before publication. This project found no single named repository or individual publisher whose isolated error constitutes the finding; every anomaly is a systemic, template-family-level pattern. Naming specific repositories elsewhere in this paper for methodological transparency is distinct from a disclosure obligation, which this paper's own framing (staleness, not negligence) does not attribute to any individual publisher's fault.

4.8 Analytical resolution: the limits of hash-based divergence detection

For Qwen2.5-7B-Instruct, the audit found a real hash-level difference between two groups of quantizers, but direct rendering of both templates against the same non-tool-calling prompt produced byte-identical output. The entire difference lives inside a Jinja conditional branch that a non-tool-calling prompt never exercises. Hash-based divergence detection can flag templates that are byte-different but behaviorally identical for a given prompt shape; the static audit's raw prevalence number is therefore an upper bound on behaviorally relevant divergence, not a direct measure of it.

5 Results

5.1 Overall divergence prevalence

Table 4 summarizes the static audit across all five anchors. The gold-standard, canonical-verified claim is that 34.5% (30/87) of GGUF repositories diverge from the chat template their model developer actually published, computed across Mistral, Qwen2.5, and Phi-3. The secondary, explicitly unverified claim is modal-disagreement of 44.8% (Llama-3.1) and

Table 4: Static-audit divergence prevalence by anchor.

Anchor	n	Prevalence	Canon.?
Mistral-7B-v0.3	35	51.4%	Yes
Qwen2.5-7B	17	23.5%	Yes
Phi-3-mini-4k	35	22.9%	Yes
Llama-3.1-8B	29	44.8%	No (gated)
gemma-2-9b-it	27	11.1%	No (gated)

Table 5: Per-anchor Wilson 95% CIs on divergence prevalence.

Anchor	Prevalence	95% CI
Mistral-7B-v0.3	51.4%	[35.6, 67.0]
Llama-3.1-8B	44.8%	[28.4, 62.5]
Qwen2.5-7B	23.5%	[9.6, 47.3]
Phi-3-mini-4k	22.9%	[12.1, 39.0]
gemma-2-9b-it	11.1%	[3.9, 28.1]

11.1% (gemma), reported separately since canonical ground truth was unavailable for these two gated families. Per-anchor prevalence spans a four-fold range, from 11.1% to 51.4%.

Table 5 reports Wilson 95% CIs per anchor rather than pooled, since the corpus contains only five effective clusters and a pooled interval would understate true uncertainty. Several intervals do not overlap: gemma's sits entirely below Mistral's and Llama-3.1's.

5.2 Popularity stratification

The pre-registered hypothesis was that divergence concentrates in the unpopular long tail. The data does not support this. Pooled, high-popularity repositories diverge at 34.7% (26/75) and long-tail at 29.4% (20/68), nearly identical. Per-anchor, Table 6 shows an inconsistent pattern: Mistral's high-popularity repositories diverge at 73.3% versus 35.0% long-tail, the opposite of Llama-3.1 (33.3% vs. 57.1%). The finding is that divergence prevalence is model-family-idiosyncratic, not popularity-driven.

5.3 Upstream drift: staleness, not negligence, for one anchor

For Mistral, 16 of 18 divergent repositories (88.9%) were created before the earliest repository using the correct template appeared, including the three highest-download quantizers used in this paper's behavioral pilot, all created within about 20 hours of each other in May 2024, roughly five months before the earliest correct-template repository in October 2024. This is ecosystem propagation lag, not publisher carelessness. For the other four anchors, staleness explains 0% of measured divergence; every divergent repository postdates the correct template's earliest known appearance. Mistral's 51.4% static divergence rate and its 88.9% staleness figure are one finding (a divergence rate) plus one explanatory mechanism, not two independent results.

Table 6: Popularity-stratified divergence by anchor.

Anchor	High-pop.	Long-tail	Direction
Mistral-7B-v0.3	73.3%	35.0%	High worse
Llama-3.1-8B	33.3%	57.1%	Tail worse
Phi-3-mini-4k	26.7%	20.0%	High worse
Qwen2.5-7B	26.7%	0.0%	High worse (n=2)
gemma-2-9b-it	13.3%	8.3%	High worse

5.4 Behavioral confirmation

Behavioral testing was attempted across all five anchors: one statistically confirmed effect, one real but underpowered effect, two ceiling effects, and one anomaly resolved analytically.

Mistral-7B-v0.3 (confirmed). The flagged divergence, used by the popular quantizers MaziyarPanahi, lmstudio-community, and SanctumAI, raises an exception when a system-role message is present. At n=21, the content-delivery effect was +23.8pp but statistically indistinguishable from noise. Scaled to n=56, the effect grew to +35.7pp, with non-overlapping Wilson 95% CIs ([60.4, 83.0] vs. [26.0, 50.6]), the first statistically significant result in this project.

Phi-3-mini-4k (real, underpowered). The flagged divergence silently drops the system-role content with no error. At n=21, the effect was +14.3pp; scaled to n=56, it shrank to +5.4pp, CIs heavily overlapping. Phi-3's baseline refusal (76–87%) leaves far less headroom than Mistral's (37–73%), and the power calculation indicates a 5–10pp effect needs $n \approx$195–784, well beyond the n=56 budget sized for the larger effect n=21 had suggested.

gemma-2-9b-it (ceiling effect). Gemma's modal, official template rejects system-role content by design, a deliberate developer choice; a minority publisher template works around it. The content-delivery effect is 0.0pp: the model refuses at 95–100% regardless, leaving no headroom.

Llama-3.1-8B (ceiling on content; novel shape finding). Both observed templates deliver system-role content, so content-delivery is 0.0pp, another ceiling effect. What differs is verbosity: the minority template carries tool-calling boilerplate. This isolates a pure template-shape effect, +9.5pp, simpler template showing higher refusal, not yet statistically confirmed at this sample size but a genuine novel result.

Qwen2.5-7B (resolved analytically). The flagged divergence lives entirely inside an unexercised tool-calling branch; direct rendering confirmed byte-identical output for the tested prompt shape, carrying no behavioral consequence for ordinary use.

Table 7 summarizes all five. Across the four anchors with

Table 7: Behavioral confirmation summary.

Anchor	Baseline	Effect	Confirmed?
Mistral-7B-v0.3	37.5%	+35.7pp	**Yes**
Phi-3-mini-4k	82.1%	+5.4pp	No
gemma-2-9b-it	95.2%	0.0pp	Ceiling
Llama-3.1-8B	95.2%	0.0/+9.5pp[†]	Ceiling
Qwen2.5-7B	—	N/A	Resolved

[†]content / shape effect

Table 8: ModelScope external-validity check.

Anchor	*n*	MS diverge	HF prevalence
Mistral-7B-v0.3	14	71.4%	51.4%
Qwen2.5-7B	18	44.4%	23.5%
Phi-3-mini-4k	16	68.8%	22.9%
Pooled	**48**	**60.4%**	**34.5%**

a directly measured effect, magnitude tracks inversely with each anchor's own baseline refusal rate; this pattern rests on four points, two of which are ceiling effects structurally uninformative about the correlation, and is offered as a hypothesis for future work, not a headline finding.

5.5 External-validity check: a second, independent registry

Table 8 reports the ModelScope check for the three canonical-verified anchors. ModelScope-measured prevalence is higher than Hugging Face's for all three anchors, a different magnitude from a genuinely different sampling frame; what matters is that the phenomenon replicates on both registries, not that the numbers match. A ModelScope repository published under the `microsoft` organization name for Phi-3 carries a template diverging from canonical, confirmed as a real content difference; this project has no independent way to verify that organization is authentically developer-controlled the way this project verified Hugging Face's own `microsoft/Phi-3-mini-4k-instruct` repository, so the upstream-drift interpretation for this specific case is held with lower confidence than the Mistral finding.

6 Discussion

6.1 Answering the long-tail objection

Isn't this just a long-tail problem? The data says no: high-popularity repositories diverge at 34.7% and long-tail at 29.4%, nearly identical, and for Mistral the popular repositories are dramatically the worse ones (73.3% vs. 35.0%), precisely because popular quantizers were made early and never refreshed against an upstream revision the less-popular repositories already incorporated. Popularity does not track template fidelity in either direction, and a practitioner cannot substitute download-count triage for actually checking the template. A user who has internalized the ordinary open-source-hygiene advice, prefer the widely-used package, will apply exactly the wrong heuristic for at least one of this paper's five anchors.

6.2 Practitioner guidance

A systematic review of the diffusion-of-innovation literature identifies complexity as one of the strongest predictors of resistance to an otherwise-beneficial change [3]. Security rarely offers the kind of sweeping benefit that justifies a complex fix, so the practical lesson is to favor the simplest, most explainable fix first.

The simplest fix: verify a GGUF's embedded chat template against its declared base model's current canonical template before serving it, surfacing a visible divergence flag. It requires no judgment about why a divergence exists, only whether one does. A second, slightly more involved fix targets the staleness mechanism: a registry-side signal, "this base model's template has changed since your quantization was created," using timestamps the registry already stores, though the specific relationship (quantization X declares base model Y, and Y's template has since changed) may need to be modeled as a first-class queryable object if it is not already. A third, complementary step belongs to loader maintainers: surface the raw chat template a user is about to run, even in minimal form, rather than hiding it behind a model-selection dropdown. None of the three requires a registry or loader maintainer to build a new detection system or adjudicate publisher intent, which is deliberate: per the diffusion-of-innovation evidence above [3], a recommendation demanding a large engineering investment tends to sit unimplemented.

6.3 What this paper does not claim

This paper does not claim that any model developer's safety training is inadequate, compromised, or worse than another's. Every behavioral effect reported is a within-model comparison, not a between-model ranking. The observation that effect magnitude tracks each anchor's own baseline refusal rate is a hypothesis for future, properly-powered testing, not a general safety ranking. The central claim stays narrow throughout: whether a downloaded model's chat template matches what its developer shipped, not whether any developer's underlying safety training is adequate.

7 Conclusion

Ask a user why they trust the model they just downloaded, and they will point to the weights. Almost nobody points to the chat template, because almost nobody knows it is there.

That is the trust boundary this paper measures: it sits inside a binary header, nobody verifies it against what the developer actually shipped, and 34.5% of the time, for the three model families where we could check, it has quietly drifted. The risk is not where intuition says to look; popularity is not a reliable signal in either direction. And where that drift is testable against real model behavior, its consequences are bounded by the model's own baseline safety training, not uniform across model families.

Future work. A wider corpus, targeting the originally-planned N≈150–300 more precisely, would tighten confidence margins further. Automated semantic-diff tooling comparing Jinja2 against Go `text/template` would allow Ollama's own multi-publisher registry to be measured natively. Longitudinal tracking of a fixed set of repositories over time, complementary to "The Moving Target" [1], would distinguish a one-time snapshot from an ongoing pattern. The hedged baseline-refusal-rate observation above is nominated as a hypothesis for a properly-powered, prospectively-designed study, not asserted as this paper's own finding.

Ethics Considerations

This study involves no human subjects. All data consists of publicly hosted, developer- and third-party-published model configuration metadata (chat templates, system prompts, sampling parameters) and model-generated text produced by running published, publicly available quantized models against a fixed, previously-published prompt set (HarmBench [6]). No private, personal, or sensitive user data was collected or processed. This is defensive security research measuring existing, already-public model behavior under different configurations of publicly available software; it does not develop or release a new attack. Any concrete safety-relevant anomaly discovered would be disclosed to the relevant publisher or registry before publication (Section 4); no such individually-attributable anomaly was found, since every finding is a systemic, template-family-level pattern rather than one publisher's isolated mistake. Naming specific high-download repositories (Section 5) is done for methodological transparency, not as an implicit accusation, consistent with this paper's own finding that the mechanism in those cases is upstream propagation lag rather than negligence.

Open Science Appendix

The full corpus (143 Hugging Face repository identifiers and extracted GGUF metadata), all analysis scripts, the custom GGUF binary header parser, and the Stage-2 behavioral pilot transcripts and labeled classifications for all five anchors are available at an anonymized mirror: `https://anonymous.4open.science/r/llm-registry-integrity-artifact-1437/`. This mirror is registered under conference ID `SEC27`, giving it an expiration date (Dec. 26, 2026) that covers this cycle's full review timeline through the shepherd-approval deadline (Dec. 15, 2026). The mirror and its contents have been checked for author name, affiliation, and other identifying detail. The de-anonymized source repository will be made available at camera-ready.

References

[1] Zhicao Fan, Yanhang Li, Zexin Zhuang, Xian Sun, and Yingshuo Wang. The moving target: A longitudinal audit of trustworthiness drift across twelve checkpoints of open-source chat LLMs. arXiv, 2026.

[2] Ariel Fogel, Omer Hofman, Eilon Cohen, and Roman Vainshtein. Inference-time backdoors via hidden instructions in LLM chat templates. arXiv, 2026.

[3] Trisha Greenhalgh, Glenn Robert, Fraser Macfarlane, Paul Bate, and Olivia Kyriakidou. Diffusion of innovations in service organizations: Systematic review and recommendations. *The Milbank Quarterly*, 82(4):581–629, 2004.

[4] Hugging Face. Use Ollama with any GGUF model on Hugging Face hub. `https://huggingface.co/docs/hub/ollama`. Accessed 2026-07-29.

[5] JFrog Security Research. GGUF-SSTI. Vulnerability disclosure, CVE-2024-34359.

[6] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[7] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs. arXiv, 2024.